Practical – 11

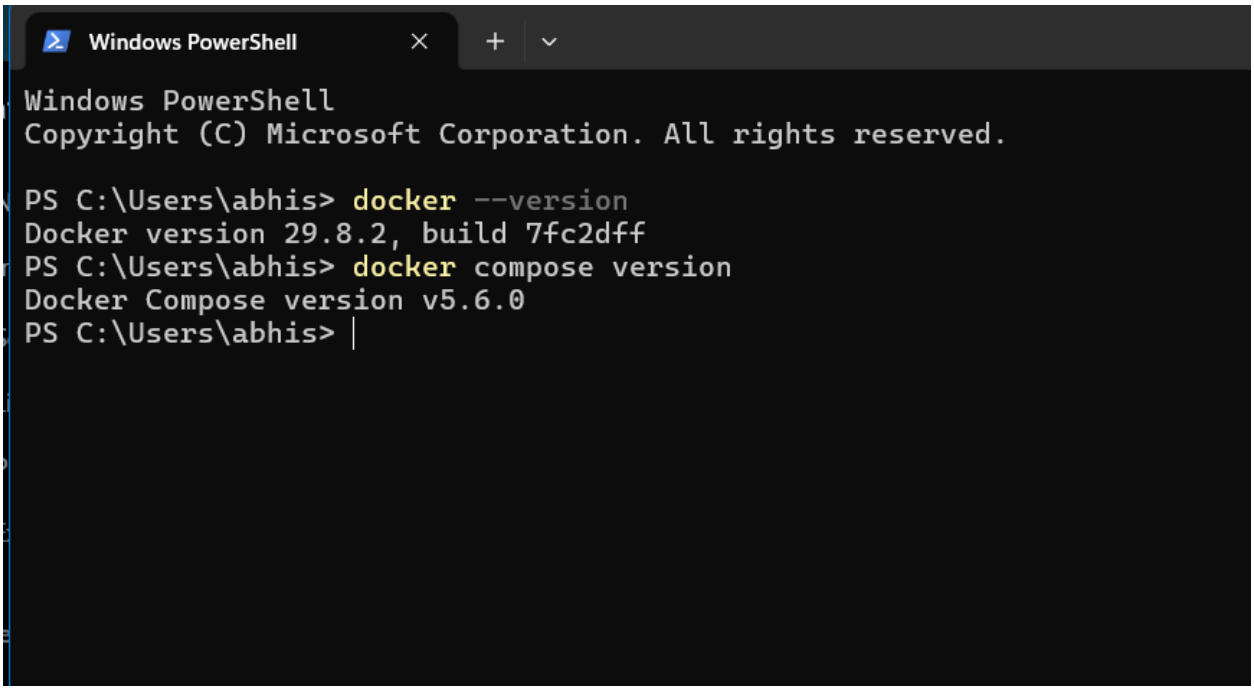
Aim:

To containerize the full-stack application using Docker and orchestrate services using Docker Compose.

Content:

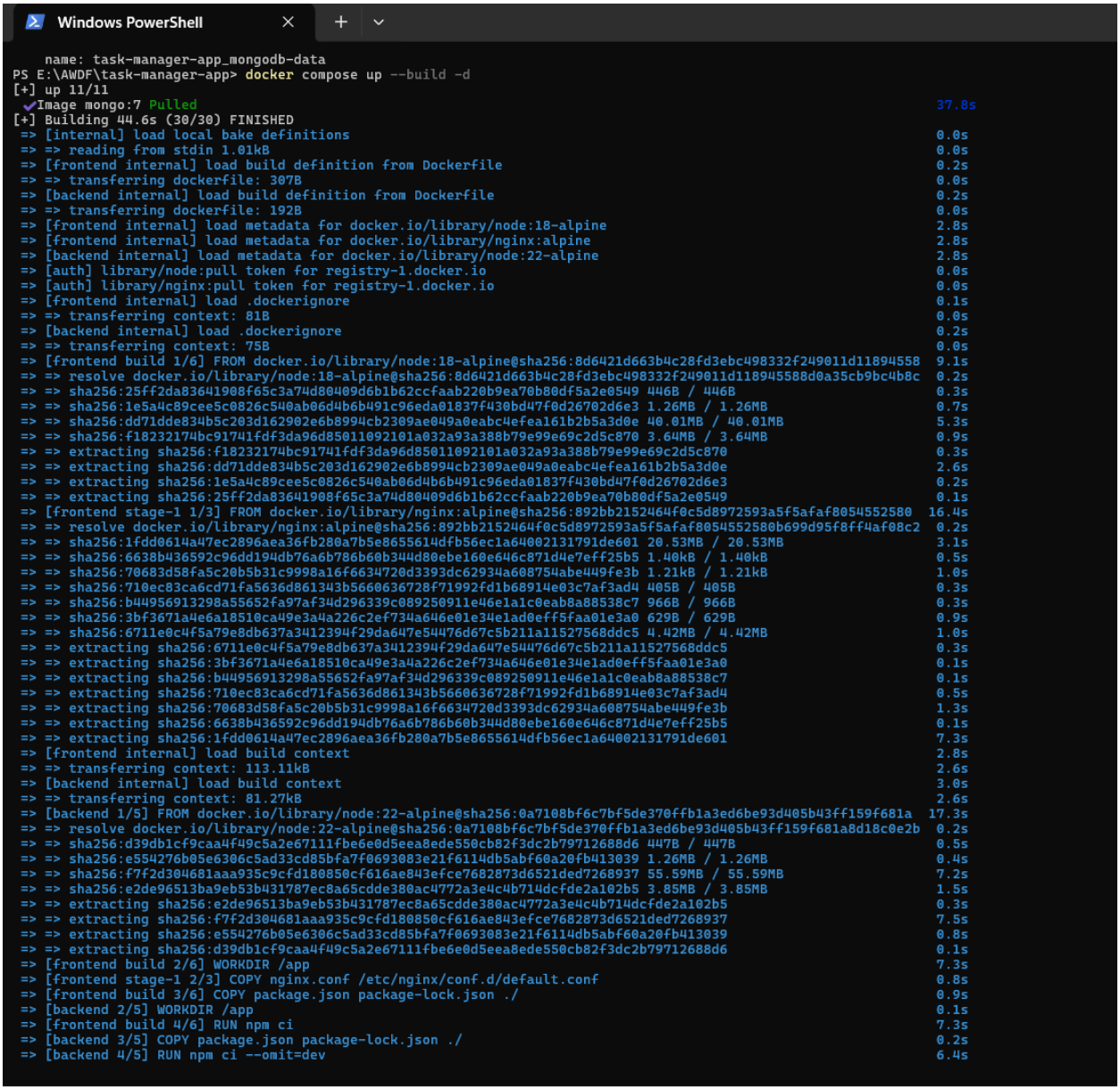
After completing this practical, I will be able to deploy a Task Management application using Docker and Docker Compose. I will learn how to create Dockerfiles for the frontend and backend, configure MongoDB as a database service, and connect all services using Docker Compose. I will also learn how to build Docker images, run containers, and check their status. This practical will help me understand containerization and deploy a full-stack application in a consistent and reliable environment.

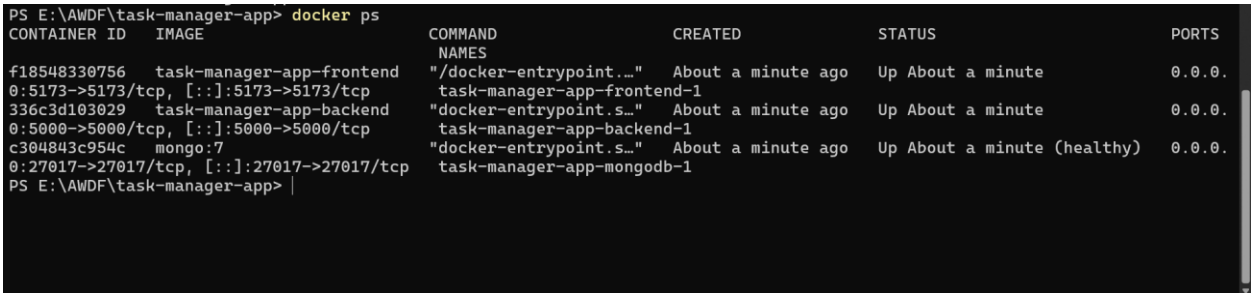
Screen Shot:

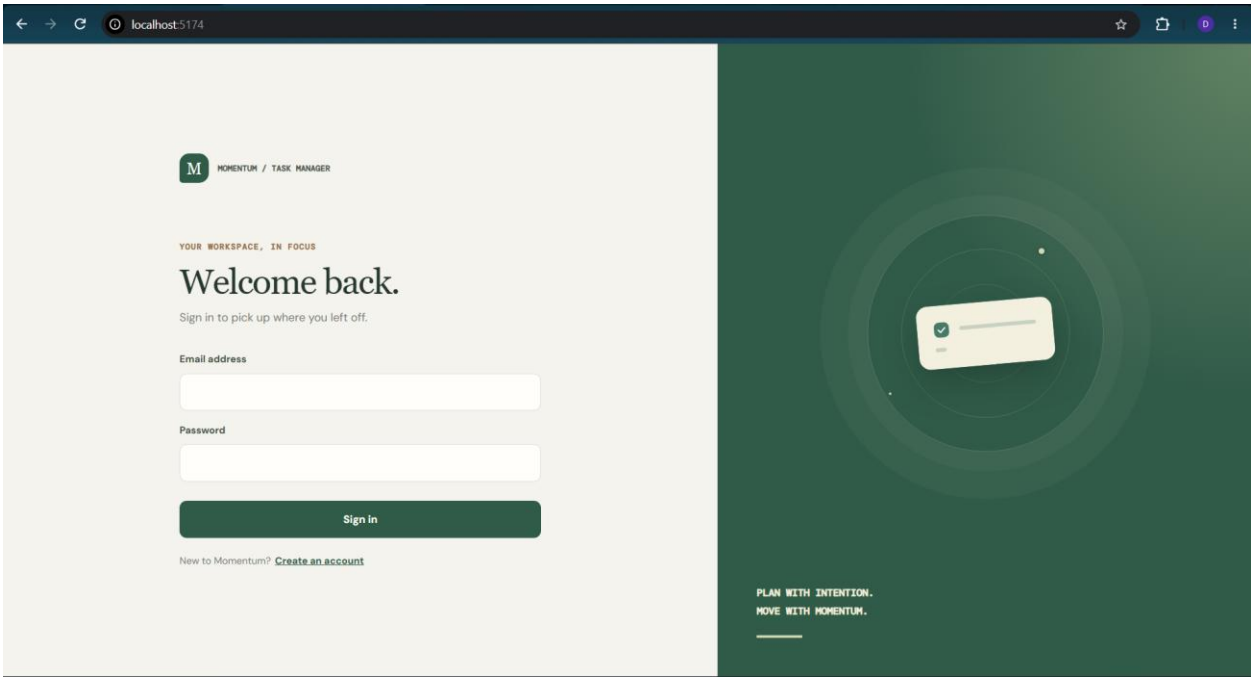


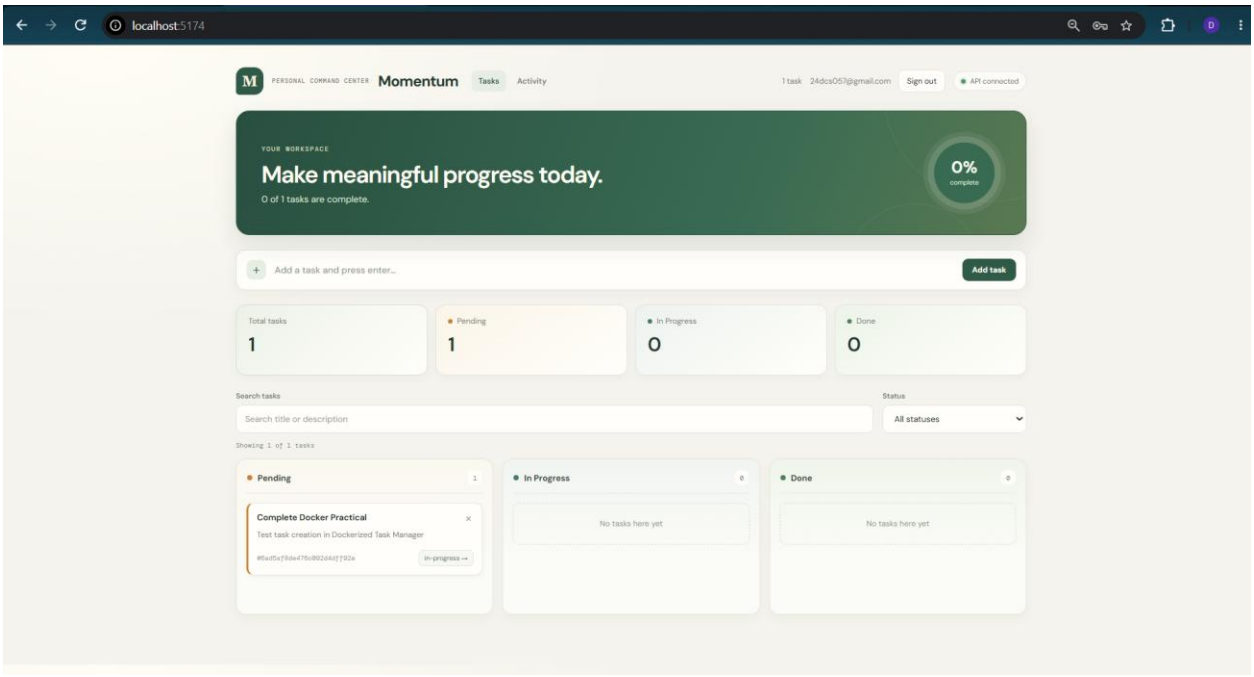
Windows PowerShell

```
PS C:\Users\abhis> cd E:\AWDF\task-manager-app
PS E:\AWDF\task-manager-app> docker compose config
name: task-manager-app
services:
  backend:
    environment:
      JWT_SECRET: task-manager-dev-only-secret
      MONGO_URI: mongodb://mongodb:27017/taskmanager
      PORT: "5000"
    networks:
      app-network: null
    build:
      context: E:\AWDF\task-manager-app\backend
      dockerfile: Dockerfile
    depends_on:
      mongodb:
        condition: service_healthy
        required: true
    ports:
      - mode: ingress
        target: 5000
        published: "5000"
        protocol: tcp
  frontend:
    networks:
      app-network: null
    build:
      context: E:\AWDF\task-manager-app\frontend
      dockerfile: Dockerfile
    depends_on:
      backend:
        condition: service_started
        required: true
    ports:
      - mode: ingress
        target: 5173
        published: "5173"
        protocol: tcp
  mongodb:
    image: mongo:7
    networks:
      app-network: null
    volumes:
      - type: volume
        source: mongodb-data
        target: /data/db
        volume: {}
    healthcheck:
      test:
        - CMD
        - mongosh
        - --quiet
        - --eval
        - db.adminCommand('ping').ok
      timeout: 5s
      interval: 5s
      retries: 10
      start_period: 10s
    ports:
      - mode: ingress
        target: 27017
        published: "27017"
        protocol: tcp
networks:
  app-network:
    name: task-manager-app_app-network
    driver: bridge
```


Windows PowerShell
name: task-manager-app_mongodb-data
PS E:\AWDF\task-manager-app> docker compose up --build -d
[+] up 11/11
✔Image mongo:7 Pulled 37.8s
[+] Building 44.6s (30/30) FINISHED
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 1.01kB 0.0s
=> [frontend internal] load build definition from Dockerfile 0.2s
=> => transferring dockerfile: 307B 0.0s
=> [backend internal] load build definition from Dockerfile 0.2s
=> => transferring dockerfile: 192B 0.0s
=> [frontend internal] load metadata for docker.io/library/node:18-alpine 2.8s
=> [frontend internal] load metadata for docker.io/library/nginx:alpine 2.8s
=> [backend internal] load metadata for docker.io/library/node:22-alpine 2.8s
=> [auth] library/node:pull token for registry-1.docker.io 0.0s
=> [auth] library/nginx:pull token for registry-1.docker.io 0.0s
=> [frontend internal] load .dockerignore 0.1s
=> => transferring context: 81B 0.0s
=> [backend internal] load .dockerignore 0.2s
=> => transferring context: 75B 0.0s
=> [frontend build 1/6] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588 9.1s
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8c 0.2s
=> => sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549 446B / 446B 0.3s
=> => sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3 1.26MB / 1.26MB 0.7s
=> => sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 40.01MB / 40.01MB 5.3s
=> => sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870 3.64MB / 3.64MB 0.9s
=> => extracting sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870 0.3s
=> => extracting sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 2.6s
=> => extracting sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3 0.2s
=> => extracting sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549 0.1s
=> [frontend stage-1 1/3] FROM docker.io/library/nginx:alpine@sha256:892bb2152464f0c5d8972593a5f5afaf8054552580 16.4s
=> => resolve docker.io/library/nginx:alpine@sha256:892bb2152464f0c5d8972593a5f5afaf8054552580b699d95f8ff4af08c2 0.2s
=> => sha256:1fdd0614a47ec2896aea36fb280a7b5e8655614dfb56ec1a64002131791de601 20.53MB / 20.53MB 3.1s
=> => sha256:6638b436592c96dd194db76a6b786b60b344d80ebe160e646c871d4e7eff25b5 1.40kB / 1.40kB 0.5s
=> => sha256:70683d58fa5c20b5b31c9998a16f6634720d3393dc62934a608754abe449fe3b 1.21kB / 1.21kB 1.0s
=> => sha256:710ec83ca6cd71fa5636d861343b5660636728f71992fd1b68914e03c7af3ad4 405B / 405B 0.3s
=> => sha256:b44956913298a55652fa97af34d296339c089250911e46e1a1c0eab8a88538c7 966B / 966B 0.3s
=> => sha256:3bf3671a4e6a18510ca49e3a4a226c2ef734a646e01e34e1ad0eff5faa01e3a0 629B / 629B 0.9s
=> => sha256:6711e0c4f5a79e8db637a3412394f29da647e54476d67c5b211a11527568ddc5 4.42MB / 4.42MB 1.0s
=> => extracting sha256:6711e0c4f5a79e8db637a3412394f29da647e54476d67c5b211a11527568ddc5 0.3s
=> => extracting sha256:3bf3671a4e6a18510ca49e3a4a226c2ef734a646e01e34e1ad0eff5faa01e3a0 0.1s
=> => extracting sha256:b44956913298a55652fa97af34d296339c089250911e46e1a1c0eab8a88538c7 0.1s
=> => extracting sha256:710ec83ca6cd71fa5636d861343b5660636728f71992fd1b68914e03c7af3ad4 0.5s
=> => extracting sha256:70683d58fa5c20b5b31c9998a16f6634720d3393dc62934a608754abe449fe3b 1.3s
=> => extracting sha256:6638b436592c96dd194db76a6b786b60b344d80ebe160e646c871d4e7eff25b5 0.1s
=> => extracting sha256:1fdd0614a47ec2896aea36fb280a7b5e8655614dfb56ec1a64002131791de601 7.3s
=> [frontend internal] load build context 2.8s
=> => transferring context: 113.11kB 2.6s
=> [backend internal] load build context 3.0s
=> => transferring context: 81.27kB 2.6s
=> [backend 1/5] FROM docker.io/library/node:22-alpine@sha256:0a7108bf6c7bf5de370ffb1a3ed6be93d405b43ff159f681a 17.3s
=> => resolve docker.io/library/node:22-alpine@sha256:0a7108bf6c7bf5de370ffb1a3ed6be93d405b43ff159f681a8d18c0e2b 0.2s
=> => sha256:d39db1cf9caa4f49c5a2e67111fbe6e0d5eea8ede550cb82f3dc2b79712688d6 447B / 447B 0.5s
=> => sha256:e554276b05e6306c5ad33cd85bfa7f0693083e21f6114db5abf60a20fb413039 1.26MB / 1.26MB 0.4s
=> => sha256:f7f2d304681aaa935c9cfd180850cf616ae843efce7682873d6521ded7268937 55.59MB / 55.59MB 7.2s
=> => sha256:e2de96513ba9eb53b431787ec8a65cdde380ac4772a3e4c4b714dcfde2a102b5 3.85MB / 3.85MB 1.5s
=> => extracting sha256:e2de96513ba9eb53b431787ec8a65cdde380ac4772a3e4c4b714dcfde2a102b5 0.3s
=> => extracting sha256:f7f2d304681aaa935c9cfd180850cf616ae843efce7682873d6521ded7268937 7.5s
=> => extracting sha256:e554276b05e6306c5ad33cd85bfa7f0693083e21f6114db5abf60a20fb413039 0.8s
=> => extracting sha256:d39db1cf9caa4f49c5a2e67111fbe6e0d5eea8ede550cb82f3dc2b79712688d6 0.1s
=> [frontend build 2/6] WORKDIR /app 7.3s
=> [frontend stage-1 2/3] COPY nginx.conf /etc/nginx/conf.d/default.conf 0.8s
=> [frontend build 3/6] COPY package.json package-lock.json ./ 0.9s
=> [backend 2/5] WORKDIR /app 0.1s
=> [frontend build 4/6] RUN npm ci 7.3s
=> [backend 3/5] COPY package.json package-lock.json ./ 0.2s
=> [backend 4/5] RUN npm ci --omit=dev 6.4s


PS E:\AWDF\task-manager-app> docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
f18548330756 task-manager-app-frontend "/docker-entrypoint.…" About a minute ago Up About a minute 0.0.0.0:5173->5173/tcp, [::]:5173->5173/tcp task-manager-app-frontend-1
336c3d103029 task-manager-app-backend "docker-entrypoint.s…" About a minute ago Up About a minute 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp task-manager-app-backend-1
c304843c954c mongo:7 "docker-entrypoint.s…" About a minute ago Up About a minute (healthy) 0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp task-manager-app-mongodb-1
PS E:\AWDF\task-manager-app>


localhost:5174
MOMENTUM / TASK MANAGER
YOUR WORKSPACE, IN FOCUS
Welcome back.
Sign in to pick up where you left off.
Email address
Password
Sign in
New to Momentum? Create an account
PLAN WITH INTENTION.
MOVE WITH MOMENTUM.


localhost:5174
PERSONAL COMMAND CENTER
Momentum
Tasks
Activity
Sign out
API connected
YOUR WORKSPACE
Make meaningful progress today.
0 of 1 tasks are complete.
0%
complete
Add a task and press enter...
Add task
Total tasks
1
Pending
1
In Progress
0
Done
0
Search tasks
Search title or description
Status
All statuses
Showing 1 of 1 tasks
Pending
In Progress
Done
Complete Docker Practical
Test task creation in Dockerized Task Manager
No tasks here yet
No tasks here yet

Conclusion:

This practical demonstrates the implementation of containerization in a Task Management application using Docker, Docker Compose, React, Node.js, Express.js, and MongoDB. It includes creating Dockerfiles for the frontend and backend, configuring MongoDB as a database service, and connecting all services using Docker Compose. It also covers building Docker images, running containers, checking container status, and testing the application through the browser. This practical helps understand containerization and makes application deployment easier, more consistent, and reliable.